Analítica de Datos

Clase 12. Optimización y despliegue de modelos

Comparar hiperparámetros y utilizar el modelo para generar nuevas predicciones

Luis Miguel Gómez Meneses
Universidad Cooperativa de Colombia
Ingeniería de Software — Séptimo semestre

6 de noviembre de 2026

Duración de la sesión: 3 horas continuas

Cronograma de la sesión

Duración total: 180 minutos. Tres horas continuas de teoría y práctica.

Tiempo	Actividad	Modalidad
15 min	Entrenamiento, sobreajuste y subajuste	Repaso
30 min	Optimización de hiperparámetros	Teoría
40 min	Laboratorio 24: comparar configuraciones	Práctica
30 min	Introducción al despliegue de modelos	Teoría
50 min	Laboratorio 25: guardar, cargar y utilizar el modelo	Práctica
15 min	Interpretación de resultados y conclusiones	Cierre
180 min	**Total de la sesión**	

Del entrenamiento a las nuevas predicciones

Entrenar ajusta los pesos y los bias. Para utilizar la red, también necesitamos seleccionar su configuración y preparar cómo recibirá datos nuevos.

Optimización de hiperparámetros

- Comparar distintas configuraciones de la red.
- Elegir utilizando los resultados de validación.
- Evaluar el modelo elegido en la prueba reservada.

Despliegue básico

- Guardar y cargar el modelo entrenado.
- Aplicar a las entradas nuevas el mismo preprocesamiento aprendido en entrenamiento.
- Ofrecer predicciones desde una interfaz sencilla.

Resultado esperado

Un modelo evaluado y listo para generar predicciones con datos nuevos.

Entrenamiento, validación y prueba

Separar los datos permite distinguir entre **aprender**, **elegir una configuración** y **evaluar el modelo final**.

Entrenamiento

Ajusta los **pesos y los bias** de la red.

El optimizador utiliza los gradientes para actualizar estos parámetros.

Validación

Permite **comparar hiperparámetros**.

También ayuda a detectar sobreajuste y decidir cuándo detener el entrenamiento.

Prueba

Evalúa el **modelo ya seleccionado**.

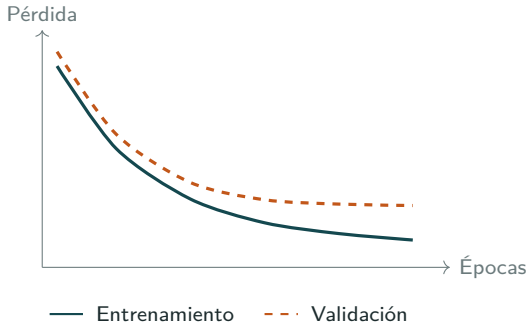
Utiliza datos reservados que no participaron en el entrenamiento ni en la selección.

Criterio para comparar modelos

Elegimos la configuración con **validación**. Reservamos la **prueba** para la evaluación final.

Lectura de las curvas de aprendizaje

Las curvas muestran cómo cambia la pérdida a lo largo del entrenamiento. Una pérdida menor representa un mejor ajuste según la función elegida.



Aspectos que observamos

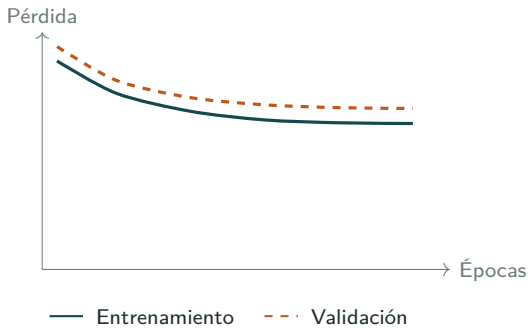
- El nivel de ambas pérdidas.
- Su tendencia al aumentar las épocas.
- La separación entre las curvas.

Una brecha pequeña también puede aparecer cuando ambos resultados son malos.

Curvas conceptuales. La escala de la pérdida depende del problema y de la función utilizada.

Subajuste: aprendizaje insuficiente

El modelo obtiene un desempeño insuficiente tanto en entrenamiento como en validación.



Posibles causas

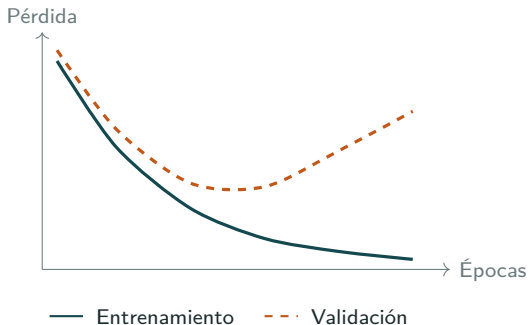
- Capacidad insuficiente de la red.
- Entrenamiento demasiado corto.
- Una configuración que dificulta aprender.
- Entradas poco informativas.

La cercanía entre las curvas no demuestra un buen modelo.

Esquema conceptual. El nivel considerado alto debe juzgarse respecto al problema y a un modelo de referencia.

Sobreajuste: mejora que no se conserva

El modelo mejora en entrenamiento, pero esa mejora deja de trasladarse a validación.



Señales habituales

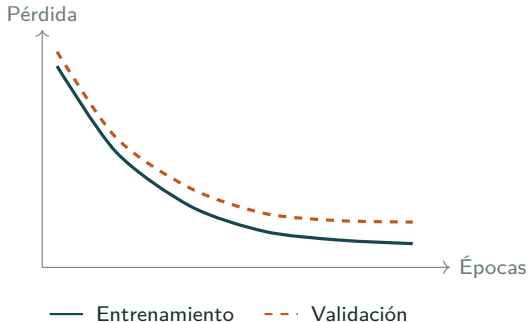
- La pérdida de entrenamiento continúa bajando.
- La pérdida de validación aumenta de forma sostenida.
- La separación entre ambas crece.

El último estado de la red puede ser peor que uno anterior.

Curvas conceptuales. Una fluctuación aislada no basta para diagnosticar sobreajuste.

Un ajuste que generaliza

Buscamos buen desempeño en entrenamiento y validación, con una diferencia razonable entre ambos.



Interpretación

Las dos pérdidas descienden y se estabilizan en valores útiles para el problema.

Las curvas orientan el diagnóstico. La evaluación final se realiza después en prueba.

Curvas conceptuales. Con Dropout o aumento de datos, la pérdida de entrenamiento puede superar la de validación.

Decisiones a partir del diagnóstico

Situación	Revisión inicial	Cambio que podemos probar
Subajuste	Capacidad y evolución de la pérdida	Más entrenamiento, otra tasa o mayor capacidad.
Sobreajuste	Brecha y deterioro en validación	Parada temprana, regularización o menor capacidad.
Pérdida inestable	Escala de entradas y tasa de aprendizaje	Corregir el procesamiento o reducir la tasa.

Comprobar el efecto

Cada cambio es una hipótesis. Su utilidad se verifica en validación y se registra en la comparación.

Parámetros e hiperparámetros

Parámetros

Pesos y bias que se ajustan durante el entrenamiento.

$$w_{\text{nuevo}} = w_{\text{actual}} - \eta \frac{\partial L}{\partial w}$$

El optimizador determina las actualizaciones a partir de los gradientes.

Hiperparámetros

Configuraciones que elegimos y comparamos mediante validación.

- Capas, neuronas y activaciones.
- Tasa de aprendizaje y tamaño del lote.
- Regularización y criterio de parada.

La ecuación ilustra descenso de gradiente. Optimizadores como Adam utilizan una regla de actualización adicional.

Espacio de búsqueda del laboratorio

Definimos de antemano qué valores vamos a explorar y qué condiciones permanecerán iguales.

Configuración	Valores propuestos
Neuronas en una capa oculta	32 y 128
Tasa de aprendizaje	0.01, 0.001 y 0.0001
Activación oculta / salida	ReLU / softmax con 10 salidas
Optimizador / tamaño del lote	Adam / 128 ejemplos
Límite de entrenamiento	12 épocas con parada temprana

Dos tamaños por tres tasas: seis configuraciones.

Valores propuestos para la práctica. La mejor combinación se determinará al ejecutar el experimento.

Estrategias de búsqueda

Estrategia	Cómo compara	Uso y limitación
Manual	Prueba configuraciones elegidas por el analista.	Facilita explorar efectos, pero depende del recorrido elegido.
Cuadrícula	Evalúa todas las combinaciones de una lista de valores.	Ordenada, aunque el costo crece al añadir opciones.
Aleatoria	Muestrea configuraciones dentro de rangos definidos.	Controla el número de intentos, sin cubrir todo el espacio.

Hoy utilizaremos una cuadrícula pequeña de seis combinaciones.

Fuente: scikit-learn, ajuste de hiperparámetros.

Condiciones para una comparación justa

- Utilizar las mismas particiones de entrenamiento y validación.
- Mantener el mismo procesamiento de entradas y la misma métrica.
- Inicializar una red nueva para cada configuración.
- Aplicar el mismo límite de épocas y criterio de parada.
- Registrar semillas, configuración, tiempo y resultados.

Variabilidad del entrenamiento

Una semilla ayuda a reproducir el experimento. Si dos candidatos quedan muy cerca, conviene repetirlos con varias semillas antes de afirmar una ventaja.

Criterio de selección y métricas

Criterio principal

Elegiremos la menor **pérdida de validación** del modelo restaurado por la parada temprana.

La pérdida considera las probabilidades asignadas a las clases correctas.

Información complementaria

- Exactitud en entrenamiento y validación.
- Curvas y brecha entre conjuntos.
- Épocas utilizadas y tiempo.
- Cantidad de parámetros.

$$\text{Exactitud} = \frac{\text{predicciones correctas}}{\text{total de ejemplos evaluados}}$$

Exactitud corresponde a accuracy. Precision es otra métrica y se calcula respecto a las predicciones de una clase.

Parada temprana y recuperación del mejor estado

La parada temprana vigila la validación y evita continuar indefinidamente cuando deja de mejorar.

```
import tensorflow as tf

parada = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", mode="min", patience=3,
    restore_best_weights=True
)
```

- `patience=3`: tolera tres épocas consecutivas sin mejora.
- `restore_best_weights=True`: recupera los pesos del mejor punto.

Crear un callback nuevo para cada entrenamiento. La época final ejecutada puede diferir de la mejor época.

Fuente: Keras, EarlyStopping.

Ejemplo de selección entre candidatos

EJEMPLO DIDÁCTICO CON VALORES FICTICIOS

Resultados calculados después de restaurar el mejor estado de cada candidato.

Modelo	Neuronas	Tasa	Acc. ent.	Pérdida val.
A	32	0.01	0.990	0.190
B	128	0.001	0.982	**0.115**
C	128	0.01	0.999	0.240
D	32	0.0001	0.890	0.370

Elección según el criterio definido

Seleccionamos B porque tiene la menor pérdida de validación. La mayor exactitud de entrenamiento de C no lo convierte en el mejor candidato.

Validación fija y validación cruzada

Partición fija

Cada configuración se entrena y se compara usando una única separación común de datos.

Es la estrategia de la sesión por su costo y facilidad de seguimiento.

Validación cruzada

Se repite el entrenamiento sobre distintas particiones y se resumen los resultados de validación.

Permite examinar la variación entre particiones y requiere más entrenamientos.

Seis candidatos con cinco particiones requieren 30 entrenamientos.

La prueba final permanece reservada. Si hay grupos, sitios o fechas, la partición debe respetar esa estructura.

Fuente: scikit-learn, validación cruzada.

Evaluación final del modelo elegido

La configuración se fija antes de consultar el resultado de prueba.

- Calcular pérdida y exactitud sobre la prueba reservada.
- Examinar la matriz de confusión para identificar clases problemáticas.
- Revisar algunos aciertos y errores concretos.
- Informar las limitaciones y la diferencia frente a validación.

Evitar seleccionar con la prueba

Si cambiamos el modelo para mejorar ese resultado, el conjunto ya está influyendo en la selección y deja de funcionar como evaluación independiente.

Una red con el reescalado integrado

La misma estructura se utiliza al crear cada candidato del laboratorio 24.

```
import tensorflow as tf
def crear_red(neuronas, tasa):
    red = tf.keras.Sequential([
        tf.keras.Input(shape=(28, 28)),
        tf.keras.layers.Rescaling(1./255),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(neuronas, activation="relu"),
        tf.keras.layers.Dense(10, activation="softmax")
    ])
    red.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=tasa),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"])
    return red
```

Las etiquetas son enteros de 0 a 9. La función crea una red nueva y sin entrenar.

Laboratorio 24: comparar configuraciones

Objetivo

Comparar seis redes densas para clasificar dígitos de MNIST y justificar la configuración elegida con resultados de validación.

Entradas

Imágenes de 28×28 píxeles en escala de grises, con valores entre 0 y 255.

Salida

Una de diez clases: dígitos del 0 al 9.

Fuente: Keras, MNIST.

Experimento

Una capa oculta con 32 o 128 neuronas y tres tasas de aprendizaje.

Duración

40 minutos de trabajo guiado.

Laboratorio 24: datos y procedimiento

Tiempo	Trabajo
5 min	Cargar MNIST y fijar una muestra estratificada de 15 000 ejemplos del conjunto original de entrenamiento.
5 min	Separar esa muestra en 12 000 para entrenar y 3 000 para validar. Mantener los 10 000 oficiales de prueba reservados.
20 min	Entrenar los seis candidatos con los mismos datos, máximo 12 épocas, lote de 128 y parada temprana.
10 min	Comparar, seleccionar, evaluar el elegido en prueba y redactar la conclusión.

Los 45 000 ejemplos restantes no se utilizan en esta práctica. La muestra reducida limita el costo. Si se reduce el presupuesto, debe aplicarse por igual a todos los candidatos.

Laboratorio 24: resultados que se registran

- Tabla de las seis configuraciones con pérdida de validación, exactitud, épocas y tiempo.
- Curvas de entrenamiento y validación de los candidatos relevantes.
- Identificación del modelo elegido y explicación del criterio.
- Métricas de prueba y matriz de confusión del modelo seleccionado.

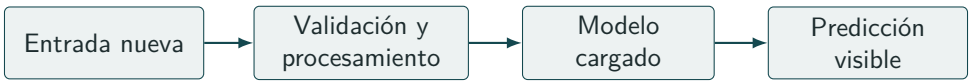
Continuidad con el laboratorio 25

Conservar el modelo seleccionado en la variable `mejor_modelo`, junto con la configuración y los resultados.

Para comparar entrenamiento y validación al final, evaluar ambos conjuntos con el mismo modelo restaurado y en modo de inferencia.

Despliegue de un modelo analítico

Desplegar consiste en integrar el modelo en un entorno donde pueda recibir entradas y devolver predicciones de forma utilizable.



Alcance de la sesión

Construiremos una demostración funcional con una interfaz sencilla. Guardar el archivo es un paso previo a poner el modelo en uso.

Entrenamiento e inferencia

Aspecto	Entrenamiento	Inferencia
Propósito	Ajustar el modelo.	Obtener una predicción.
Información	Entradas y etiquetas conocidas.	Nuevas entradas.
Pesos y bias	Se actualizan.	Permanecen fijos.
Cálculo	Forward, pérdida y gradientes.	Forward con parámetros aprendidos.

Uso desde una aplicación

Cada solicitud utiliza el modelo cargado. No es necesario entrenarlo otra vez para predecir.

Formato de las nuevas entradas

El modelo espera la misma representación que recibió durante el entrenamiento.

Característica	**Acuerdo para esta práctica**
Contenido	Un único dígito centrado.
Tamaño y canales	28×28 píxeles, escala de grises.
Intensidades externas	Valores entre 0 y 255.
Convención visual	Trazo claro sobre fondo oscuro, como MNIST.
Forma de un lote	$(n, 28, 28)$, donde n es el número de imágenes.

Una imagen con otro tamaño se rechazará en la demostración. Fotografías arbitrarias requieren un procesamiento y una evaluación adicionales.

Procesamiento consistente de las entradas

El reescalado integrado se aplica tanto al entrenar como al predecir.

$$x_{\text{reescalado}} = \frac{x_{\text{original}}}{255}$$

Valor original	Operación	Valor reescalado
0	$0/255$	0
128	$128/255$	≈ 0.502
255	$255/255$	1

Una sola transformación

Si el modelo contiene `Rescaling(1./255)`, las entradas se entregan en 0–255. Dividirlas antes produciría un segundo reescalado incorrecto.

Fuente: Keras, Rescaling.

Guardar y cargar el modelo

El formato `.keras` conserva la arquitectura, los pesos y la configuración del modelo.

```
import tensorflow as tf

mejor_modelo.save("modelo_mnist.keras")

modelo = tf.keras.models.load_model(
    "modelo_mnist.keras",
    compile=False
)
```

Cargar para inferencia

`compile=False` es suficiente para obtener predicciones. Para continuar el entrenamiento se requiere configurar la compilación correspondiente.

Fuente: Keras, guardado y carga de modelos.

Comprobar que la carga conserva las predicciones

Utilizamos exactamente las mismas entradas antes y después de cargar el archivo.

```
import numpy as np
lote = x_prueba[:5]
antes = mejor_modelo(lote, training=False).numpy()
despues = modelo(lote, training=False).numpy()
np.testing.assert_allclose(
    antes, despues, rtol=1e-5, atol=1e-6
)
```

Interpretación

La comprobación acepta diferencias numéricas pequeñas. Si falla, hay que revisar el archivo cargado, las entradas y el procesamiento.

La verificación comprueba consistencia de la serialización. La calidad predictiva se determina con la evaluación del modelo.

Función para predecir una imagen

La función recibe una imagen PIL y utiliza el modelo ya cargado.

```
import numpy as np
import gradio as gr

def predecir(imagen):
    if imagen is None:
        return {}
    imagen = imagen.convert("L")
    if imagen.size != (28, 28):
        raise gr.Error("Use una imagen de 28 x 28 pixeles.")
    x = np.asarray(imagen, dtype="float32")[None, ...]
    p = modelo(x, training=False).numpy()[0]
    return {str(i): float(p[i]) for i in range(10)}
```

`[None, ...]` añade la dimensión del lote. El reescalado permanece dentro del modelo.

Interfaz de predicción con Gradio

La interfaz conecta una entrada de imagen con la función y muestra las clases predichas.

```
demo = gr.Interface(
    fn=predecir,
    inputs=gr.Image(
        type="pil", image_mode="L", sources=["upload"],
        label="Digito claro sobre fondo oscuro: 28 x 28"
    ),
    outputs=gr.Label(num_top_classes=3),
    title="Clasificacion de digitos MNIST"
)

demo.launch(share=False)
```

Ejemplo para ejecución local. Primero deben existir `modelo` y `predecir`. En Colab, el acceso puede requerir un enlace temporal según el entorno.

Fuente: Gradio, Interface.

Interpretación de la salida del modelo

Softmax produce diez valores que suman aproximadamente uno. Elegimos la clase con el mayor valor.

EJEMPLO DIDÁCTICO

Clase	Valor asignado
7	0.82
1	0.12
9	0.04
Resto de clases, en conjunto	0.02

Alcance de la interpretación

El modelo elegiría 7. Un valor alto puede acompañar una predicción equivocada y no garantiza que el modelo esté bien calibrado.

Archivos y dependencias de la aplicación

Elemento	Contenido
`modelo_mnist.keras`	Modelo elegido con reescalado integrado.
`app.py`	Carga del modelo, función de predicción e interfaz.
`requirements.txt`	Versiones probadas de TensorFlow, NumPy, Pillow y Gradio.
Configuración y resultados	Valores utilizados, semilla, métricas y formato de entrada.

Entorno reproducible

Registrar las versiones que funcionaron y comprobar la carga en el entorno de destino antes de distribuir la aplicación.

En Colab, Gradio puede instalarse con `%pip install gradio`. Una instalación que actualiza dependencias puede requerir reiniciar el entorno.

Demostración local y publicación

Demostración de la sesión

El proceso se ejecuta en el equipo o entorno del estudiante.

La interfaz permite comprobar el funcionamiento completo con entradas conocidas y nuevas.

Servicio publicado

Requiere un entorno disponible, recursos de ejecución y configuración de acceso.

También necesita seguimiento de errores, tiempo de respuesta y cambios en los datos.

Un enlace temporal de demostración depende del proceso que lo sirve. No equivale a un servicio permanente.

Errores frecuentes al utilizar el modelo

Problema	Efecto	Comprobación
Forma incorrecta	La red rechaza la entrada.	Revisar lote, alto y ancho.
Reescalado duplicado	Intensidades mucho menores a las esperadas.	Confirmar dónde se divide por 255.
Fondo o trazo invertidos	La imagen difiere de MNIST.	Revisar la convención visual.
Archivo equivocado	Predicciones inconsistentes.	Verificar la versión y comparar salidas.
Datos muy diferentes	Puede producir errores aun con confianza alta.	Evaluar ejemplos representativos del uso real.

Laboratorio 25: utilizar el modelo entrenado

Objetivo

Guardar el modelo seleccionado, cargarlo y ofrecer predicciones mediante una interfaz de carga de imágenes.

- Partir del modelo obtenido en el laboratorio 24.
- Comprobar que la carga conserva las salidas.
- Utilizar imágenes de 28×28 y revisar las predicciones.
- Identificar entradas que la aplicación debe rechazar.

Duración: 50 minutos de trabajo guiado.

Laboratorio 25: secuencia de trabajo

Tiempo	Actividad
10 min	Guardar el archivo, cargarlo y verificar salidas sobre un mismo lote.
10 min	Construir la función y comprobar forma, intensidades y etiquetas.
15 min	Conectar la función con Gradio y ejecutar la interfaz.
10 min	Probar imágenes válidas y entradas de tamaño incorrecto. Registrar aciertos, errores y mensajes.
5 min	Organizar archivos y escribir las conclusiones.

Para obtener una imagen de prueba puede exportarse un ejemplo de MNIST con Pillow. Las pruebas de interfaz no se utilizarán para volver a elegir hiperparámetros.

Laboratorio 25: evidencias del funcionamiento

- Archivo `.keras` que se carga correctamente.
- Comparación numérica antes y después de cargar.
- Interfaz que recibe una imagen y muestra las clases con mayor puntuación.
- Ejemplo de una entrada válida y de una entrada rechazada.
- Registro de dependencias, formato de entrada y limitaciones.

Conclusión de la práctica

Explicar qué se guardó, cómo se procesa una nueva entrada y qué condiciones necesita el modelo para producir una predicción útil.

Comunicación de los resultados

Un reporte debe permitir entender cómo se eligió el modelo y qué tan bien funcionó en la evaluación.

Contenido	**Información que debe quedar clara**
Experimento	Datos, particiones, candidatos y condiciones comunes.
Selección	Métrica de validación y configuración elegida.
Evaluación	Resultados de prueba y errores por clase.
Uso	Formato de entrada, forma de ejecutar y salida esperada.
Limitaciones	Variabilidad, muestra utilizada y diferencias con el uso real.

Cierre de la sesión

- Las curvas ayudan a diagnosticar el aprendizaje y la generalización.
- Los hiperparámetros se comparan bajo condiciones comunes y con validación.
- La prueba se reserva para evaluar el modelo seleccionado.
- El uso del modelo exige conservar el procesamiento de las entradas.
- La interfaz permite convertir una predicción en un resultado accesible.

Continuidad del curso

Visualización de datos, indicadores KPI y dashboards para comunicar resultados y apoyar decisiones.

Documentación de apoyo

- Keras: conjunto de datos MNIST.
- Keras: parada temprana y restauración de pesos.
- Keras: reescalado de entradas.
- Keras: guardar y cargar modelos.
- scikit-learn: búsqueda de hiperparámetros.
- scikit-learn: validación cruzada.
- Gradio: interfaz para funciones de predicción.

Los enlaces son interactivos en el PDF. Las curvas y tablas marcadas como didácticas son ejemplos explicativos.